Digimarc Barcode Integration Guide

2016-03-14

This document is intended for engineers who are integrating Digimarc Barcode reading capabilities into systems with imaging cameras, typically for use in retail scanners or manufacturing inspection systems.

Additional Information

Along with this guide, we recommend reviewing the following materials available for use with the Digimarc Embedded Systems SDK:

- The SDK documentation itself, which explains key configuration features.
- Additional test material that is distributed with this guide. Some of the tests described in this guide refer to this material.

Digimarc can provide calibrated prints of some test images.

Digimarc Barcode Fundamentals

The Digimarc Barcode has its roots in the field of digital watermarking. Digimarc is a worldwide leader in this field and has invested years of research and development to deliver a robust and efficient technology to enable the retail market.

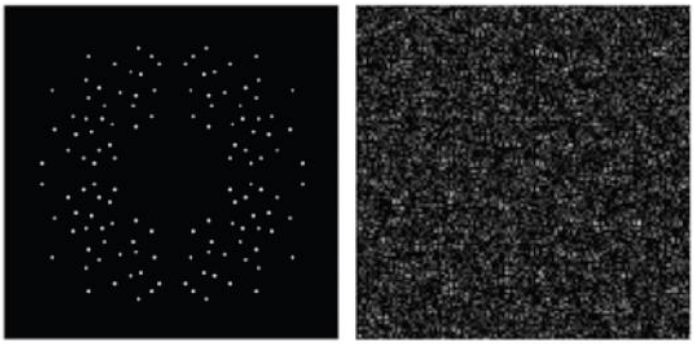
The goal of the first section of this document is to teach fundamental concepts about Digimarc Barcode so that engineering teams can understand the technology, see how it relates to their own knowledge, and make informed design decisions. Digimarc greatly respects the expertise of its partners and wants to see them succeed in ways that we cannot anticipate. The best way to do so is to share our own knowledge and to collaborate on improved solutions. We welcome your questions and input during our work together.

Components

The Digimarc Barcode is a 2D symbol that contains two components:

- Grid: This is Digimarc's term for the fixed (constant) portion of the symbol. It is defined as a constellation in the 2D Fourier domain.
- Message: This is the variable portion of the symbol. It is a spatial signal that encodes the actual data carried by the symbol. We also refer to it as the payload. The Message is encoded with robust error detection and correction techniques.

The Grid and Message are added together to define a 128 x 128 element Tile. You can picture these two components separately like this:



During the detection process, the detector uses correlation techniques to identify the presence of the Grid. If it is present, the measured rotation, scale, and translation values are used to spatially align with the Message.

The Message is encoded in 32 x 32 element sub-tiles, so each Tile is composed of both the Grid and a 4 x 4 matrix of Message sub-tiles. The detector does not need to see an entire Tile and only a single Message sub-tile is needed for decoding, but the detector will combine Message sub-tiles to improve decoding rates.

Another Digimarc term is the waxel, or "watermark element". The waxel is analogous to a pixel in a digital image.

- One waxel is one square element of the 128 x 128 element Tile.

- Each waxel represents a logical positive or negative change that will be encoded in the final artwork.

Representation

To apply the Digimarc Barcode to an image, the 128 x 128 Tile is spatially scaled to print at 75 waxels per inch, or 75 WPI.

- In other words, each waxel is 1/75 x 1/75 inches (13 x 13 mil, 0.338 x 0.338 mm).
- At 75 WPI, this makes each Tile 1.7 x 1.7 inches square (128 / 75 = 1.7).

Most package artwork is designed for 300 DPI printing. As a result, each element (a waxel) of the Tile is scaled to cover 4 x 4 pixels in the digital design. When printed, the Tile will then appear at 75 WPI (300 DPI / 4 dots per waxel).

If the underlying print media cannot support this resolution, a Digimarc designer will choose a different ratio. For example, many thermal printers are 203 DPI (8 mm print heads). In this case, the designer may choose a 67 WPI (3:1 ratio) or 100 WPI (2:1 ratio) Digimarc Barcode.

Color Spaces

As described earlier, each waxel represents a *logical positive or negative change* in the artwork. To create these changes, the Digimarc designer may use two different techniques. Either technique can represent the same logical Digimarc Barcode, but they affect the image in different ways.

- Luma
    - A Digimarc Barcode may be applied with a series of luminance changes. With Luma, pixels in the design are made brighter or darker to convey the positive and negative waxel values.
        - In RGB, all three color channels are changed in the same direction.
        - In CMYK, CMY can be changed in the same direction, and/or K can be changed.
        - In grayscale, each pixel is made brighter or darker.
    - An extreme example of a Luma Digimarc Barcode is when it is implemented in a thermal printer. Most thermal printers produce bitonal images, so only the negative (black) changes can be printed.
- Chroma
    - A Digimarc Barcode may be applied with differential changes in the available color channels. These differential changes are weighted such that they minimize the perceived luminance change.
        - In RGB, roughly equal but opposite changes are applied in R and G. Some signal is also in B, coherent with the R change.
        - In CMYK, roughly equal but opposite changes are applied in C and M. Very little change is made in Y. K is not changed.
        - Grayscale images cannot carry a Chroma Digimarc Barcode.

Depending on a package's design, a Digimarc designer may choose to use either or both of these techniques in an image to achieve the necessary coverage and visibility goals. The existence of these two techniques implies certain requirements for the imaging system. Alternatives and trade-offs are described in the following table.

Sensor Type	Illumination Type	
	Narrowband 660 nm (red)	Broadband (white)
Monochrome sensor	The system will see the red (RGB) or cyan (CMYK) channels. This is compatible with both Luma and Chroma techniques in Digimarc Barcode. Other narrowband wavelengths will emphasize other channels carrying Digimarc Barcode, but 660 nm is preferred.	Luma images will work well but Chroma images will not work reliably because the differential color changes will cancel each other in the sensor. Some techniques will improve Chroma performance with white illumination. Depending on the spectrum of the illumination, it may be possible to choose a warmer white (dominant components above 600 nm) or to insert an optical filter to only capture longer wavelengths (> 600 nm) or reject incoherent channels.
Color sensor	The system will see the red (RGB) or cyan (CMYK) channels. This is compatible with both Luma and Chroma techniques in Digimarc Barcode. Other narrowband wavelengths will emphasize other channels carrying Digimarc Barcode, but 660 nm is preferred.	If using all color channels, Luma and Chroma Digimarc Barcodes will work well but require more computation to process color images in both Luma and Chroma modes. Given sufficient computation, this combination can deliver the most robust Digimarc Barcode system because the color channels can be optimally combined. *Alternatively*, only a subset of channels (from an RGB sensor) can be processed as a luminance image. Depending on the dominant illumination wavelength, either R, B, R+B, or G can be used for both Luma and Chroma detection.

There are many choices in sensor and illumination capabilities and it is import to review the features of the camera system for best compatibility with Digimarc Barcode. Even within white LEDs, there are significant variations to understand. The table above discusses the most common options as they relate to Luma and Chroma images. Please discuss other approaches with Digimarc.

Design

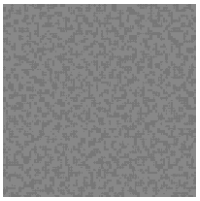
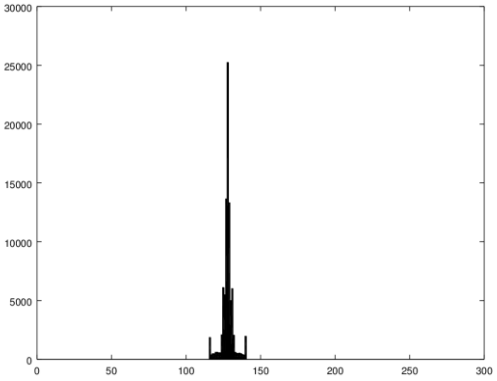
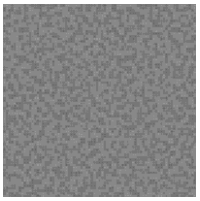
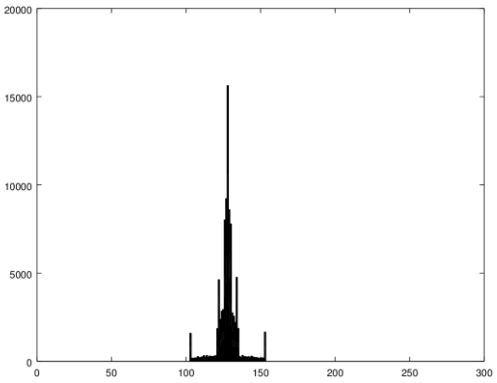
Unlike traditional identification symbols (UPC-A, Databar), Digimarc Barcode is designed to have little visual impact on a package's appearance. To achieve this, small magnitude Luma or Chroma changes

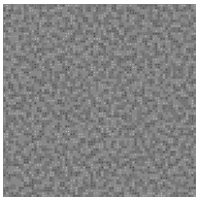
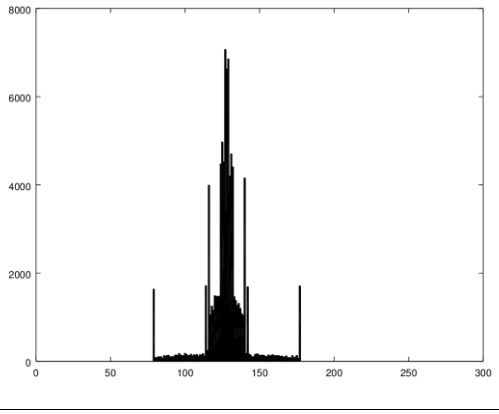
are introduced in the package's artwork. These low contrast changes can be subtle and place new requirements on retail scanners for best performance. The better an imaging system can capture these small changes, the better it will perform with Digimarc Barcode.

The following diagrams show the visual effect of Luma and Chroma types of Digimarc Barcode at low, medium, and high "strengths". By strength, we mean the magnitude of the changes that a Digimarc designer has chosen to apply to an image. At low strength, the changes in each color channel are small and less visible. At high strength, the changes in each color channel are larger and more visible.

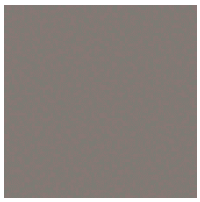
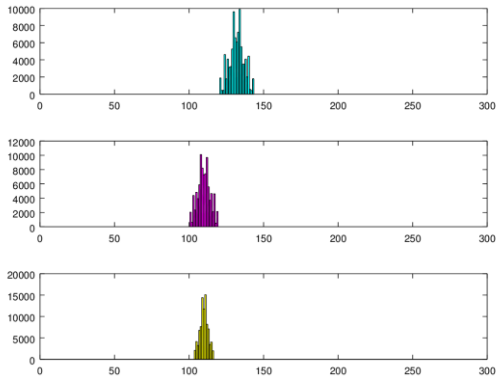
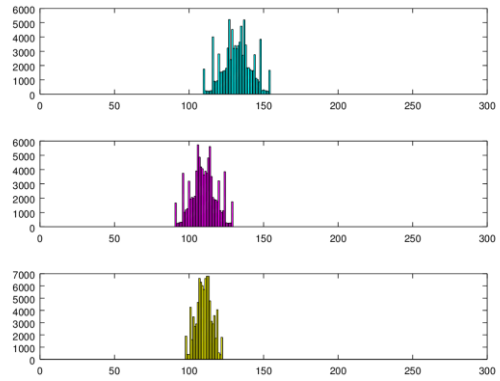
In each case, a simple RGB or CMYK gray patch is used as a reference. The examples shown here are realistic values in many designs, with strength 1 in Luma and strength 3 in Chroma being common. In the tables, SD indicates standard deviation and delta is the largest change in each patch. The histograms plot the distribution of values in the range 0-255.

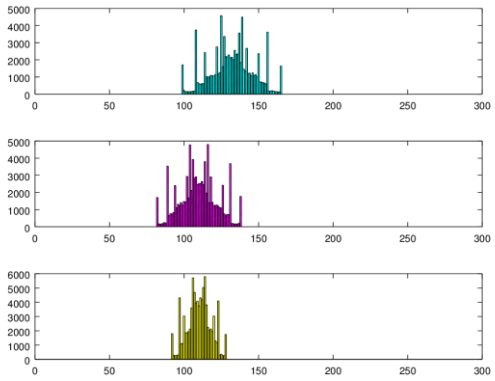
Luma Examples

Strength	Gray Patch	Histogram	Notes
1			SD = 3.73 Delta = ±12
3			SD = 7.53 Delta = ±25

5			SD = 14.94 Delta = ±49

Chroma Examples

Strength	CMYK Patch	Histogram	Notes
1			Cyan SD = 4.86 Delta = ±11 Magenta SD = 4.17 Delta = ±9 Yellow SD = 2.74 Delta = ±6
3			Cyan SD = 9.71 Delta = ±22 Magenta SD = 8.37 Delta = ±19 Yellow SD = 5.42 Delta = ±12

5			Cyan SD = 14.57 Delta = ±33 Magenta SD = 12.55 Delta = ±28 Yellow SD = 8.00 Delta = ±18

These patches are shown as digital examples to illustrate the magnitude of changes applied to package designs. In practice, another complication is the accuracy with which the artwork will be printed. Digimarc works closely with package printers to understand practical print limits and to ensure that high standards are maintained during production.

Scanner Characterization for Digimarc Barcode

This section of the document describes tests that Digimarc has found useful to compare different scanning systems and to empirically measure relevant characteristics. We respect the expertise that integrators of our technology possess, and as such, these tests are not meant to be authoritative. Other test techniques may be used to improve the same metrics.

We broadly divide these device goals into two categories:

- Capabilities
    - Capabilities are inherent image characteristics of the scanner. For example, image resolution, focal length, modulation transfer function (MTF), and noise are all characteristics of digital imaging systems. By improving these Capabilities, a scanner increases its potential to read a broader range of Digimarc Barcode items.
- Implementation
    - Implementation characteristics reflect the actual integration of Digimarc Barcode algorithms with the complete scanner system. For example, the frame rate, computational capacity, and software configuration of the Digimarc SDK all contribute to Implementation measurements.

Both of these categories are important and are independent of each other. A fast CPU with a poor image path will have a different experience than a slow CPU with a high quality image path.

Scanner Capabilities

The scanner's capabilities affect the breadth of Digimarc Barcode packages that can be detected. Because of package design constraints, some packages will have weaker characteristics that require better performance from the imaging system.

Tone Response

To successfully read Digimarc Barcode from a variety of designs, it is valuable for the system to capture images with a wide dynamic range without saturating (clipping) bright or dark areas. Once the image is saturated, the small changes in the Digimarc Barcode are lost. Likewise, a system with a narrow dynamic range will not capture small magnitude changes. Note that the absolute range of the test results is not critical, but a wider dynamic range implies that small contrast differences can be captured.

In this test, we want to confirm that the imaging system does not saturate across a range of gray patches. We also want to measure the dynamic range that the system can capture.

Steps

The following steps should be performed at three distances.

- Near - Closest expected reading distance.
- Medium – Typical/nominal reading distance.
- Far - Farthest expected reading distance.

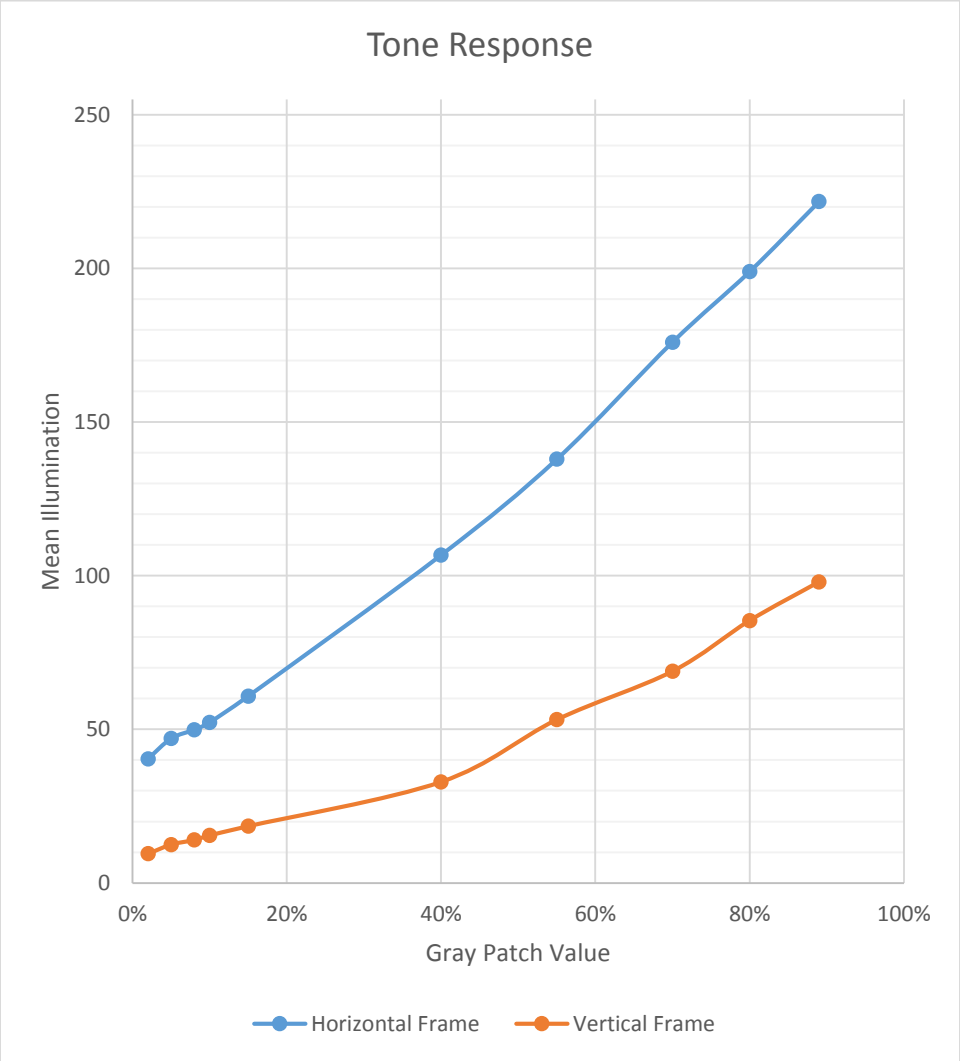
1) Locate the set of 10 gray patches included with the "Tone Response" test.
   a) These patches vary from light to dark.
      i) The patches are annotated with their reflectivity values.
   b) These patches are not Digimarc enabled.
   c) To avoid distortion and glare, is best if these patches are mounted flat on stiff cardboard.
2) With the camera system, capture a frame of each gray patch.
   a) Present each patch perpendicular to the camera's axis.
   b) Capture these images in a lossless format (PNG, BMP).
3) Review the 10 patches to find a region across all of them that is relatively free from artifacts.
   a) Avoid areas with reflection, glare, or other defects.
   b) Start with the darkest patch to choose the region. This will help to avoid glare.
4) From each frame, crop the same small region that cleanly shows the gray patch.
5) Measure the average value of each selected region. Tools such as Photoshop or GIMP can perform this measurement.
6) Graph the measured average luminance value for each capture against the reflectance value shown on each gray patch to show the tone response curve.
7) Repeat steps 2-6 at the distances described earlier, creating three data sets and three lines on the graph.

Example Output

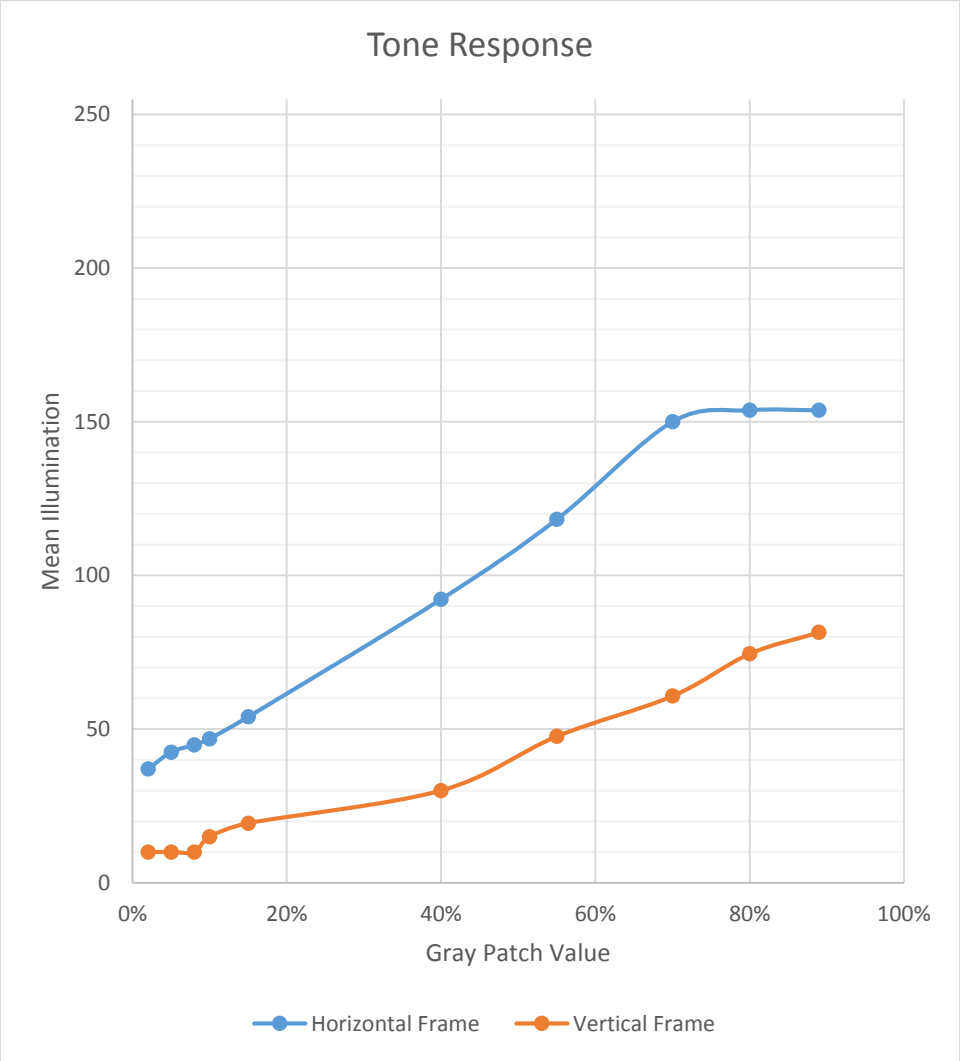
The results of this test should show a linear response from the lightest patch to the darkest patch. No saturation should be observed at either extreme. In addition, the goal is for the tone response to cover a wide dynamic range so that the small contrast changes in Digimarc Barcode can be detected.

For clarity, the following examples show data for a single tested distance instead of the three distances described in the test procedure. Data is shown for a hypothetical bioptic scanner (vertical and horizontal scanning planes). Three different scenarios are discussed.

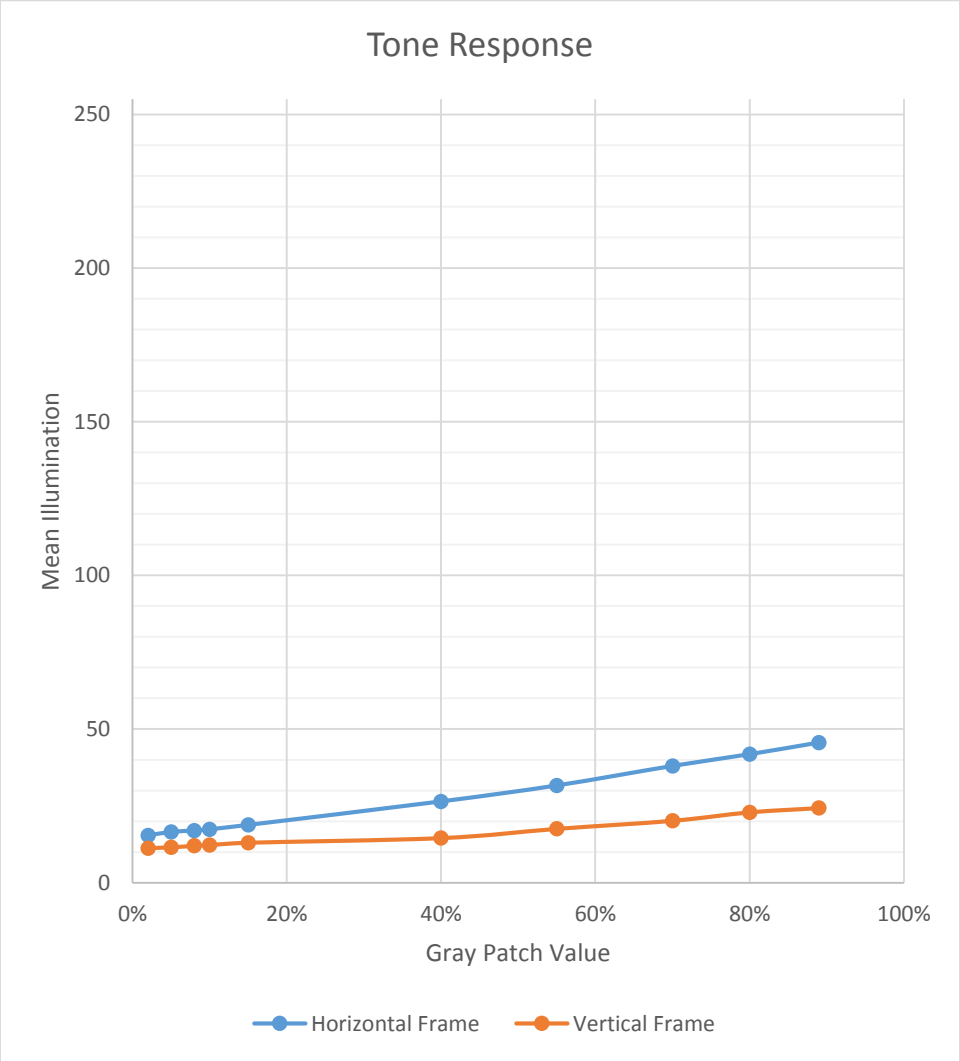
The first example shows a good response. No clipping was measured and the graph covers a good dynamic range (about 7 bits). The result would be improved if the tone response spanned an even wider range to make use of all 8 bits.



The next example shows clipping in dark colors in the vertical frame, which indicates this plane of the scanner will perform poorly with darker package designs or at farther distances. Likewise, there is clipping evident with bright colors in the horizontal frame.



The last example does not saturate, but has a shallow response (3-4 bits of dynamic range). This scanner will only be able to read packages with a strong (more visible) Digimarc Barcode. Small contrast differences will not be seen. This scanner will not perform well in most cases.



Camera Calibration

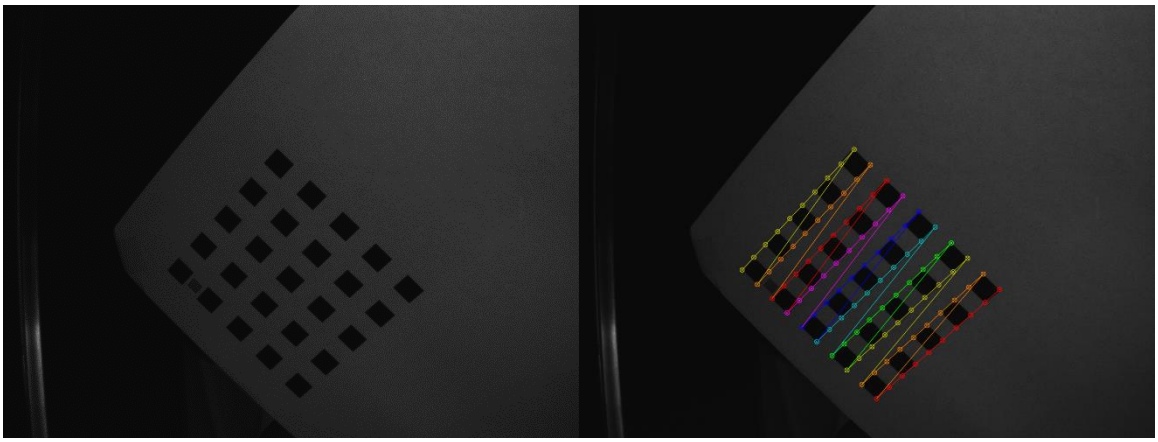
The Digimarc Embedded Systems SDK includes a feature to correct for projective distortion. To support that feature, we use a test target of a 5x5 grid of boxes to estimate characteristics of a camera. Using the test target and a tool from Digimarc, we can compute parameters needed to support the camera model in the SDK.

Steps

1) Obtain the Camera Calibration Tool from Digimarc. Alternatively, you can supply the necessary images to Digimarc for evaluation by the tool.

2) Print a copy of the Camera Calibration grid (included with the tool and the “Calibration Tests” material).
   a) It can be printed at any reasonable size to fit the scanner’s field of view.
3) Follow the instructions included with the Camera Calibration Tool to capture images and measure them.
   a) If a camera’s image is composed of multiple split views, the calibration should be done for each view to determine its orientation. In the Digimarc SDK configuration, each part of the view can be treated individually.
4) The tool computes an estimate of the camera’s focal length and optical center. The results of the tool can be used in the SDK to correct for perspective.
   a) Devices with long focal ranges (relative to the image size) will work well with the SDK’s perspective correction feature.
   b) Devices with short focal ranges (wide fields of view) present a more distorted image that will not work well with the SDK’s perspective correction feature.

Example calibration images



Example Output

The following text is the actual output of the tool after it has computed the camera’s focal length and optical center for one view. The parameters shown can be used in the SDK to correct for perspective distortion. In addition, this device’s relatively long focal length parameters will work well in perspective correction.

```
Below is SDK sample code that illustrates how the calibration information is inputted into the
PerspectiveCorrectionParams struct. The first four arguments in this struct are the X and Y focal
lengths and the X and Y components of the optical center (aka the principal point). The last two
arguments tell the detector the angles by which it should correct the image data.  Replace <tilt-
correction> and <bearing-correction> with the estimated values, printed below in the Calibration
Information, for whichever image you'd like to correct for.
Sample Code
************************************************
void BuildBlockConfiguration(Configuration& Config) {

        static const PerspectiveCorrectionParams perspectiveParams =
{2542.55, 2523.85, 707.373, 423.142, <tilt-correction>, <bearing-correction>};
        static const Block blockConfigs[] = {
                Block(<CourseScale>, <CenterX>, <CenterY>, false, &perspectiveParams),
        };

        Config.Initialize(blockConfigs, 1);
```

```
}

Calibration Information
************************************************

Estimated intrinsic camera parameters:
  X/Y focal length: 2542.55 / 2523.85
  X/Y optical center: 707.373 / 423.142


Estimated rotation angles (tilt and bearing in degrees):
VerticalBmpFull-20151019-164902.149.txt: 9.98675 275.912
VerticalBmpFull-20151019-164915.529.txt: 17.5475 17.0535
VerticalBmpFull-20151019-164920.461.txt: 37.0065 222.695
VerticalBmpFull-20151019-164938.064.txt: 27.8871 74.9557
VerticalBmpFull-20151019-164956.390.txt: 28.4511 47.197
VerticalBmpFull-20151019-165000.618.txt: 40.256 218.007
VerticalBmpFull-20151019-165019.642.txt: 12.619 258.074
VerticalBmpFull-20151019-165030.919.txt: 24.3836 60.3286
VerticalBmpFull-20151019-165054.456.txt: 42.4553 250.056
VerticalBmpFull-20151019-165107.144.txt: 28.3374 42.4864
VerticalJpeg-20151021-104355.801.txt: 14.416 265.369
VerticalParallelToCamera.txt: 15.9979 268.838

Average reprojection error (should be < 2): 1.60749 pixels

Camera intrinsic matrix:
2542.550651 0.000000 707.372674
0.000000 2523.846101 423.141947
0.000000 0.000000 1.000000
```

Spatial Resolution

The nominal feature size of Digimarc Barcode is 75 DPI (13 mil). This is the same nominal feature size as standard UPC/EAN barcodes. Because of this, it is expected that commercial barcode scanners will have more than enough resolution for Digimarc Barcode support.

Regardless, this test is used to understand the spatial resolution of the camera at different distances to determine if image downsampling is needed. The input to the Digimarc SDK can be higher resolution, but after downsampling the image is expected to be close to 75 DPI. The SDK can detect a broad range of 50-200% relative to this scale (37.5 DPI to 150 DPI), so an exact match is not needed. Usually, a single downsample ratio is sufficient to cover the entire usable distance range of the scanner. Image downsampling is configured in the SDK.

Steps

The following steps should be performed at three distances.

- Near - Closest expected reading distance.
- Medium – Typical/nominal reading distance.
- Far - Farthest expected reading distance.

1) For this test, make a color print of “Image 2” in the PDF included in the “Calibration Tests” material.
   a) Be sure it is printed at 100%, as printers often scale pages.
   b) Each square in the image should be 0.5 inch (12.7 mm) square.
   c) It is best if the print is mounted flat on stiff cardboard.
   d) Alternatively, capturing an image of a ruler is also a useful test metric.

2) With the scanner, capture a frame of “Image 2”.
   a) Present the image perpendicular to the camera’s axis.
   b) Capture the image in a lossless format (PNG, BMP).
3) Using a tool like Photoshop or GIMP, measure the number of pixels across two adjacent squares (one inch, 25.4 mm). This is the DPI result for this distance.
4) Repeat steps 2-3 for each of the distances described earlier.

Example Output



From this image, we can see that the measured resolution is 284 DPI. If this resolution was from the nominal reading distance, then the SDK should be configured to downsample by 4 to achieve an effective resolution of 284 / 4 = 71 DPI.

Divided View Size

Depending on the device, the images passed to the SDK may be either full field views or a composite of images from mirrors. This element of the system design does not have a clear positive or negative test result, but it is important to understand the system so that the SDK’s configuration can make the best use of it. Digimarc can assist with this decision.

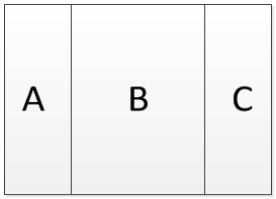
Steps

1) Measure the width and height of each divided view in a frame.
   a) Full frame captures from any previous test can be used.
   b) These divisions will affect where the SDK’s blocks should be placed. Blocks should not span divisions.
2) Collect geometry information from the manufacturer.

a) Views of each camera.
b) Direction of each camera.

Expected Output

For example, one camera may be composed of three different views gathered from mirrors.



1) Width: 1024 pixels divided into three divided views.
   a) A: 256
   b) B: 512
   c) C: 256
2) Height: 768 pixels
3) From either the manufacturer's information or the Camera Calibration test, we can determine the angle and direction of each view. The SDK can be configured to uniquely use each view.

Spectral Power Distribution

For best compatibility with Digimarc Barcode, images should be captured under specific lighting conditions. The Digimarc Barcode is applied to images as either:

- Low contrast luminance changes applied across process (CMYK) or spot color inks
- Low contrast chrominance changes differentially applied across CMY inks

To make best use of this information, the imager and its illumination should be reviewed. Certain combinations are preferred, as outlined below.

Steps

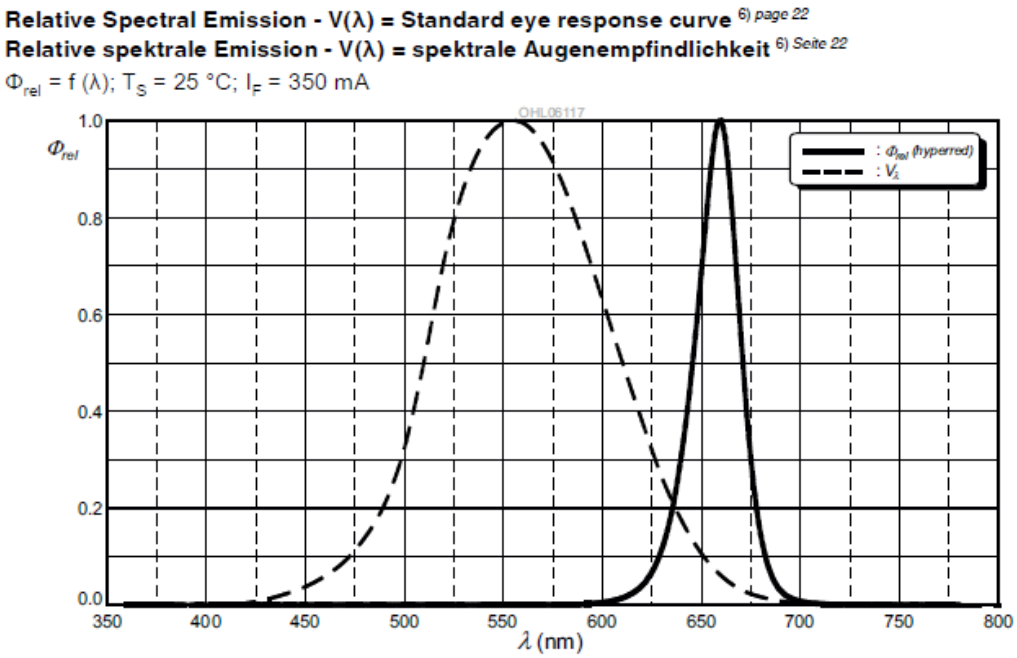
1) Review with Digimarc the datasheet for the product's illumination. Digimarc engineers can study this information and make recommendations.
2) Review the datasheet for the image sensor used in the product with Digimarc.
3) Depending on the image sensor, different choices must be made.
   a) With a monochrome imager:
      i) Digimarc Barcode is primarily designed for red 660 nm illumination.
      ii) This can be accomplished with specific LEDs or filters to remove unwanted spectral components.
   b) With a color imager:
      i) Digimarc Barcode is designed for broad spectrum white illumination. Depending on the actual illumination, different color channels will be emphasized.

ii) For best results and compatibility with Luma or Chroma Digimarc Barcodes, please consult with Digimarc on color applications.

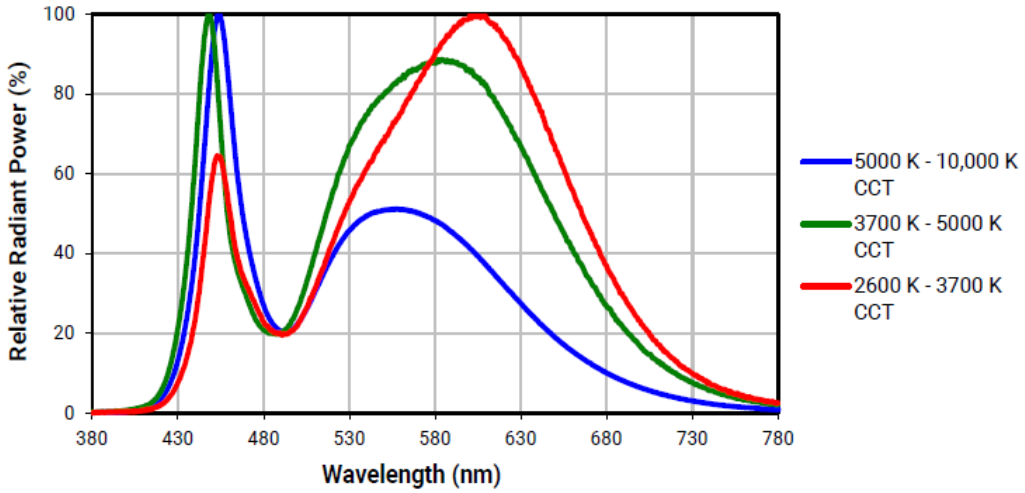
4) Sensor response
   a) Review the overlap of the spectral response of the image sensor compared to the available illumination. This overlap is typically more important with color sensors.
5) Use the data from the manufacturer to run the "Digimarc Signal Model" spreadsheet. Review the results for suitable spectral distribution.

Examples

The following chart is an example of narrowband 660 nm (red) illumination from the OSRAM LH CP7P datasheet. The V(λ) curve shows the perceived brightness to a human observer.



The next chart shows the spectral content of white LEDs in the Cree XLamp MC-E family. Depending on the color temperature, different aspects of Digimarc Barcode will be emphasized. Longpass filters or a color sensor would be recommended in this situation.



Modulation Transfer Function

Various methods exist to measure the sharpness (acuity) of optical systems. A common measurement is the modulation transfer function (MTF). Digimarc uses this measurement to make relative comparisons between systems. Other measurements such as texture acutance are of interest and may provide more detailed results.

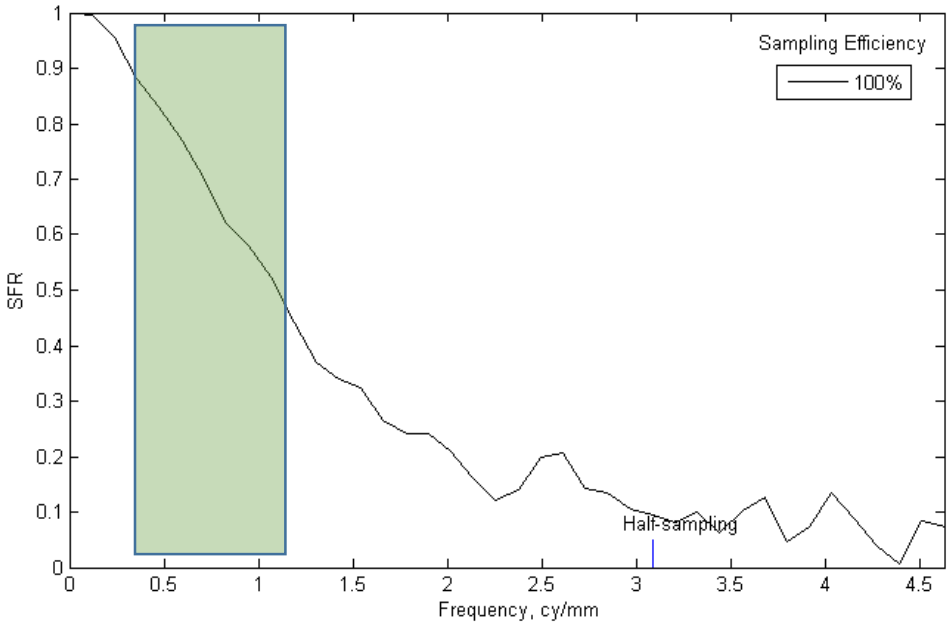
Steps

1) Take a capture of a slant edge target at the nominal usage distance.
   a) Use the ISO 12233 test target or other acuity test image.
2) Use the SFRedge tool to measure the captures, or another MTF/SFR measurement.
   a) Measure the actual spatial resolution of the captured image for use in the measurement tool.
   b) At 75 DPI, the Nyquist frequency of Digimarc Barcode is 37.5 cycles/inch.
   c) Most of the Digimarc Barcode signal exists between 25% and 75% of this rate and we expect to see strong MTF performance in this range.

Example Output

Running SFREdge will produce a chart similar to the one below. Converting cycles/inch to cycles/mm gives us:

- 37.5 cycles/inch = 1.48 cycles/mm (Nyquist rate of Digimarc Barcode)
- Lower frequency range of Digimarc barcode is 0.25 * 1.48 = 0.37 cycles/mm
- Upper frequency range of Digimarc barcode is 0.75 * 1.48 = 1.11 cycles/mm
- The area under that region of the curve is a good indicator of relative performance between different imaging systems.



Noise

The Digimarc Barcode is robust to typical noise sources, but improving SNR with lower noise characteristics is always preferred. Reducing image noise will improve detection robustness in marginal conditions or with challenging package designs. This is a relative measurement and not an absolute test goal.

At the same time, the Digimarc Barcode signal can be negatively affected by common noise-reduction algorithms so this test goal must be balanced against the detection goals of the system.

Steps

1) Take multiple static captures of the middle 55% gray patch in the Tone Response set.
   a) Place the target at the nominal distance.
   b) Do not move the test target or the imaging system during the image captures.
   c) Capture 20 frames in a lossless format.
2) Use ImageMagick (or another image tool) to compute the standard deviation of one frame compared to the average of the sequence.
   a) Average all frames
      i) convert *.bmp -evaluate-sequence mean avg_frame.bmp
   b) Compare any single frame to the averaged frame.
      i) compare -verbose -metric RMSE avg_frame.bmp 21.bmp compare.bmp

Example Output

Example ImageMagick compare output: The 0.01418 value is the standard deviation of the measured noise. Note all three channels have the same result because the grayscale images were converted to RGB BMP files.

```
compare -verbose -metric RMSE avg_frame.bmp 21.bmp compare.bmp
avg_frame.bmp BMP 1280x1024 1280x1024+0+0 8-bit sRGB 3.932MB 0.016u 0:00.077
21.bmp BMP3 1280x1024 1280x1024+0+0 8-bit sRGB 256c 1.312MB 0.063u 0:00.031
Image: avg_frame.bmp
  Channel distortion: RMSE
    red: 929.361 (0.0141811)
    green: 929.361 (0.0141811)
    blue: 929.361 (0.0141811)
    all: 929.361 (0.0141811)
avg_frame.bmp=>compare.bmp BMP 1280x1024 1280x1024+0+0 8-bit sRGB 5.243MB 0.484u 0:00.218
```

Standard Deviation - Illumination

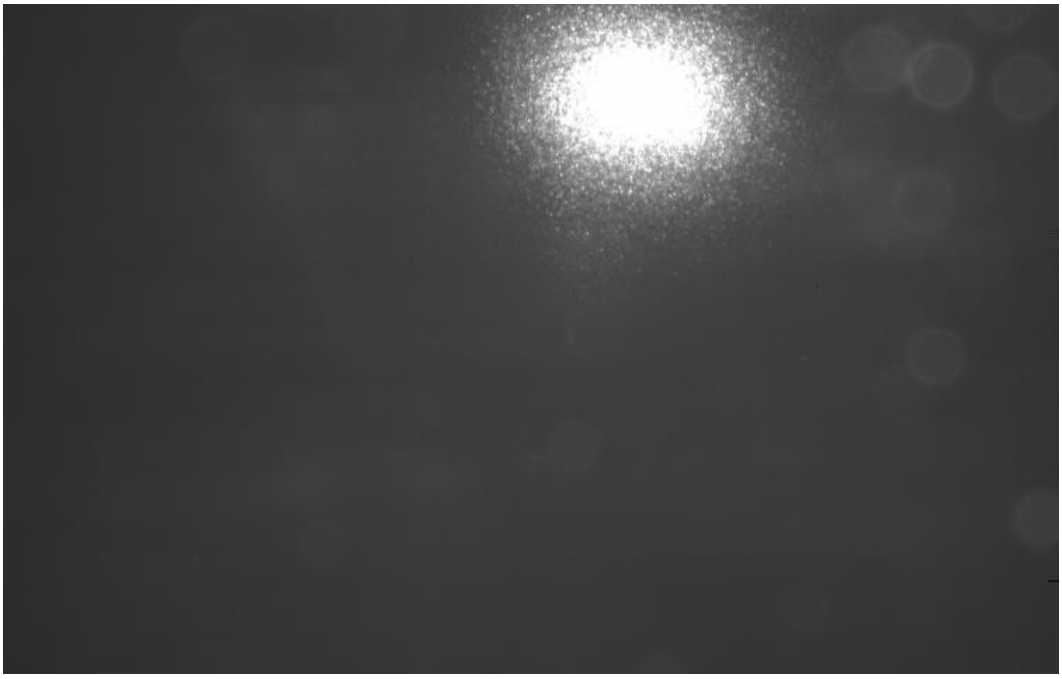
Digimarc Barcode detection is designed to work with variations in lighting but we want to avoid areas of the image with strong defects such as glare, reflections, or occlusions. Uniform lighting is preferred so that the full field of view will perform in a similar way.

Steps

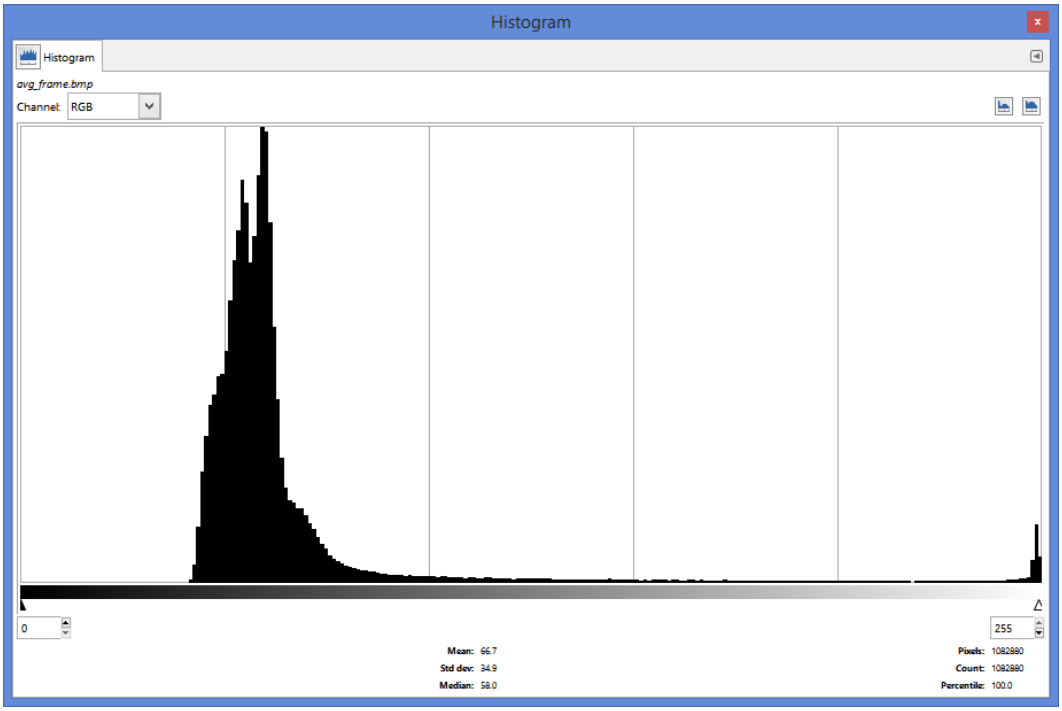
1) Review the averaged image from the noise test to identify potential areas to avoid.
2) Use the averaged frame to measure the change of illumination across the frame.
    a) A histogram is a good way of summarizing the uniformity of the image.
    b) A small standard deviation will indicate uniform illumination and SDK blocks can be placed more freely throughout the image.
    c) A large standard deviation indicates that parts of the image differ significantly and may not be useful for detection. Areas that are significantly brighter or darker than the average will offer less dynamic range.

Example Output

Here is an example averaged frame and its histogram. In general, the image is evenly illuminated. Only the area of the specular reflection should be avoided.

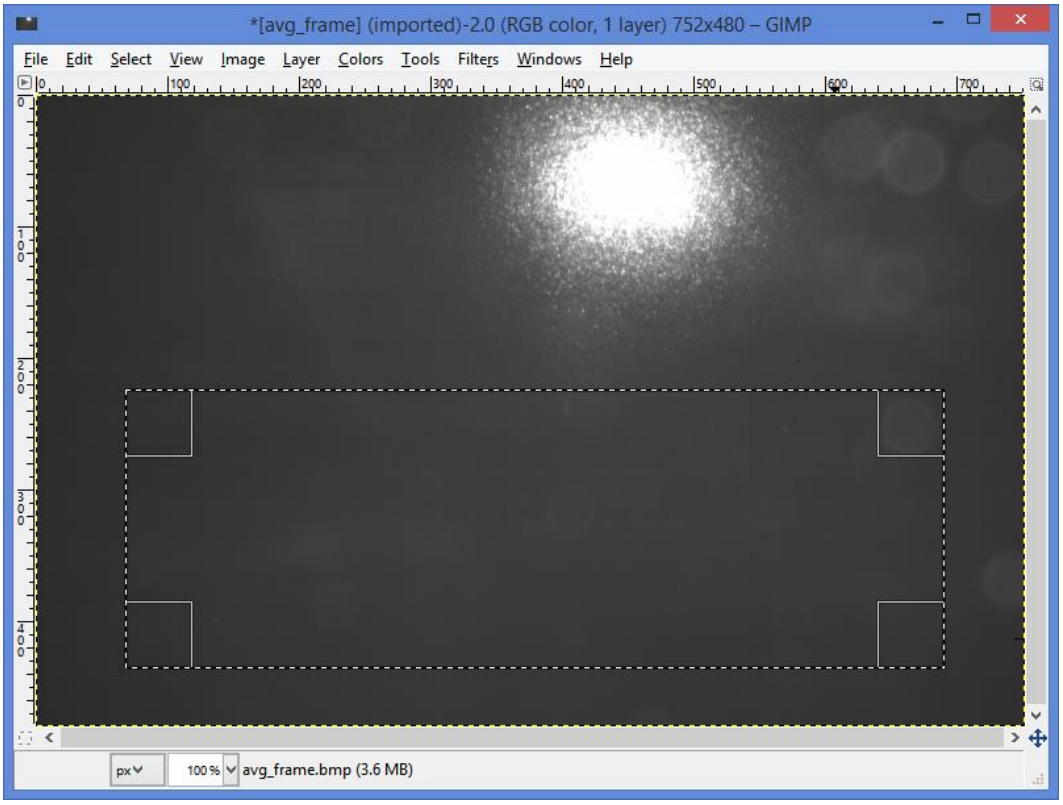


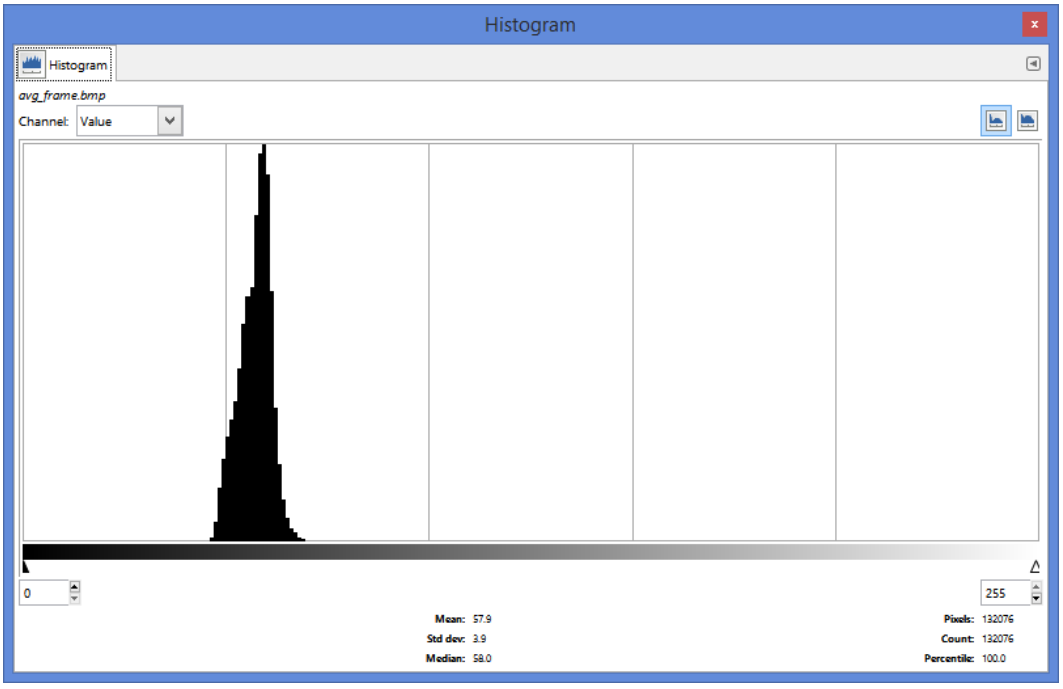
Looking at the histogram in the following picture shows that the entire image has a standard deviation of 34.9.



If we focus on the lower half of the image, as in the following picture, we find that the standard deviation is 3.9. From this review, we might favor placing SDK blocks in the lower half of the image

instead of the upper half. See the Digimarc SDK documentation for information about block configuration.





Scanner Implementation

The real value of Digimarc Barcode is determined by how it performs in practice. The previous discussions have focused on image characteristics that will improve Digimarc Barcode detection, but the final implementation will affect how a user perceives the system. In this section we review implementation metrics that Digimarc considers important.

The following topics illustrate key implementation metrics to consider. In time, Digimarc will publish suggested measurement techniques in these areas. For now, our goal is to identify these topics and explain important details in them. As a system designer, you will already have your own testing processes and metrics and we encourage you to consider the following topics in your test plan.

Speed

The time to process a single image frame is usually the most important performance metric. Digimarc has invested significant effort into optimizing its software for different processors and can continue to do so where speed is a critical factor.

The time to process a frame is divided into three logical stages:

- Image color conversion and downsampling
    - If needed, a color image is converted into a Digimarc "Luma" image and a "Chroma" image. Both output images are grayscale, but represent different weightings of the input image.
    - If needed, the image is downsampled to produce a 75 DPI image.
    - Either the entire image or active portions of the image are processed this way.
- Stage 1
    - This stage looks for the "Grid" signal discussed earlier.
    - Stage 1 runs once for each active SDK block.
    - Stage 1 computes a metric that indicates if that block is likely to contain a Digimarc Barcode. Blocks that pass this test go on to the next stage. All other blocks are dropped.
    - The false positive rate of Stage 1 is very low, approximately 1/10000.
    - Stage 1 execution time is very predictable. Every block should execute in nearly the same amount of time.
- Read Stage
    - Using data from Stage 1, the Digimarc Barcode "Message" signal is extracted and reported.
    - Extensive error detection and correction is built into Read Stage decoding.
    - Read Stage execution time is less predictable. It typically takes as much time as one block in Stage 1, but can be faster or slower.

The key insight about this architecture is that *most implementations are dominated by downsampling and Stage 1 processing of images that do not contain a Digimarc Barcode*. The Read Stage will typically run only when a Digimarc Barcode item is visible. As a result, for frame rate measurements we primarily benchmark non-Digimarc Barcode image frames. Applications in which a high percentage of image frames are expected to be readable should consider different metrics that include readable and unreadable frames.

Recommendations

1. For most implementations, we recommend a goal of sustaining at least 10 frames/second of non-Digimarc Barcode content. Some systems may work well with a lower speed and others may need faster speeds to keep up with fast moving objects.
    a. Because any part of a Digimarc Barcode package may be read, you may not need as fast of a frame rate as in traditional 1D barcode reading.
2. The number of SDK blocks and their configuration has a large impact on the per-frame time. This is the easiest way to control speed vs. detection robustness.
    a. Adding more blocks can improve coverage of the frame, but at the cost of more processing time.
    b. Each block adds a linear amount of additional processing time.
3. If available, downsampling and Digimarc-specific color conversion can be done in hardware outside of the Digimarc SDK. The SDK uses a simple NxN average for antialiasing.
4. Please consult with Digimarc on optimizations for your specific system.

Distance

The useful reading distance is also an important test parameter. The distance needed for an in-counter scanner vs. a handheld scanner will often be different, and the user of each device has different expectations. Each product has its own goals.

Recommendations

1. Test the distance limits of your system with the strongly-marked “Image 2” from the SDK. This image is not a realistic example, but it helps to determine the physical limits of your integration. If this image does not work at the desired distances, then it may be limited by an SDK misconfiguration or another of the following factors.
2. Illumination intensity varies widely with distance, so improvements in lighting or exposure will affect the farthest distances. Likewise, close objects may suffer from saturation or glare.
    a. This is especially important for handheld devices, since they may be used in a wider variety of orientations.
    b. Brighter illumination will typically increase SNR until saturation occurs.
3. When you move from testing “Image 2” to real Digimarc Barcode packages, you will primarily contend with their lower contrast (lower SNR). Improving the scanner’s performance in the areas discussed in “Scanner Capabilities” will directly affect its performance with real packages.

Perspective

Without additional corrections, the Digimarc Barcode reader is robust to +-20 degree perspective (tilt). This is a smaller arc than expected for traditional 1D barcodes and Digimarc is pursuing several improvements to this dimension of performance. SDK updates will deliver additional options.

Recommendations

1. As with “Distance”, test the limits of your system with the strongly-marked “Image 2” from the SDK. This image is a best-case example, but real-world packages will benefit from the improved metrics discussed in “Scanner Capabilities”.
    a. As with “Distance”, illumination or sensor performance will also improve perspective.

2. The SDK includes a feature for real-time perspective correction. This feature may be used statically (every frame undergoes additional processing for difference perspectives) or dynamically (different perspectives can be tried in a sequence of frames).
    a. Depending on the approach, more time will be needed per frame (static corrections) or more frames will be needed (dynamic corrections) to achieve success. The speed and usage of the scanner will help to decide the best approach.
3. Perspective correction in the SDK is more effective with systems that have long focal lengths relative to the image size.
    a. Systems with short focal lengths (wide fields of view) will show little benefit from perspective correction.

Throughput

Usually, the value of Digimarc Barcode to a POS scanner is the ability to speed up checkout efficiency by improving the ease with which cashiers can scan items. Other systems using Digimarc Barcode have other goals and different measurements are needed.

A common method to evaluate a complete POS system is to conduct one test in which throughput with only traditional 1D barcodes are measured and then another test is performed with traditional barcodes and Digimarc Barcode enabled. For example, a simple test plan to do this might include these details:

- 30 packages will be used in each run, containing X% of Digimarc Barcode items.
- At least 5 test cashiers will be evaluated.
- Each cashier will complete 10 runs of all 30 packages.
- In 5 of the runs, Digimarc Barcode will be disabled in the scanner.
- In 5 of the runs, Digimarc Barcode will be enabled in the scanner.
- Cashier instructions:
    - Consistent expectations should be given to each cashier about their role in the test.
    - Training: depending on the test goal, it may be valuable to measure cashiers who are informed about Digimarc Barcode vs. cashiers who are uninformed about Digimarc Barcode.
- The elapsed time of each run is recorded and tagged with the cashier and test setup.

This outline is provided as an example. Scanner manufacturers clearly have their own test expertise and more sophisticated test plans.

Accuracy

In other applications, such as machine vision, ease of use is not a factor. Instead, a specific identification rate must be met to satisfy the application requirements. Many aspects of this system can be defined, controlled, and measured:

- Frame rate of the capture device
- Processing rate of the CPU
- Lighting type, position, and intensity
- Items per minute inspection rate
- Item placement variation in the image frame
- Required identification rate

Depending on the system's capabilities and goals, a combination of software optimization and SDK configuration experiments will be needed to achieve success.